

Wymagania funkcjonalne i niefunkcjonalne

Temat pracy

Wykorzystanie systemu generatywnej AI do budowania ścieżki edukacyjnej dla programistów

1. Wymagania funkcjonalne (w.f.)

1.1. Funkcjonalności, które muszą pojawić się w aplikacji

Zarządzanie użytkownikami

1. System musi umożliwiać rejestrację użytkownika.
2. System musi umożliwiać logowanie i wylogowanie użytkownika.
3. System musi umożliwiać edycję podstawowych danych profilu użytkownika.
4. System musi umożliwiać zapis preferencji kursanta dotyczących nauki.

Personalizacja ścieżki edukacyjnej

5. System musi umożliwiać wybór technologii lub języka programowania do nauki.
6. System musi umożliwiać określenie poziomu zaawansowania użytkownika.
7. System musi umożliwiać określenie preferowanego rodzaju materiałów edukacyjnych (tekst, quizy, zadania praktyczne, przykłady kodu).
8. System musi umożliwiać określenie dostępnego czasu na naukę.
9. System musi generować spersonalizowaną ścieżkę edukacyjną z wykorzystaniem modelu LLM.
10. System musi dostosowywać poziom trudności materiałów do postępów użytkownika.
11. System musi umożliwiać zapis wygenerowanej ścieżki edukacyjnej.
12. System musi umożliwiać wygenerowanie co najmniej 3–4 różnych ścieżek edukacyjnych.

Generowanie materiałów edukacyjnych

13. System musi generować materiały edukacyjne dotyczące wybranej technologii programistycznej.
14. System musi generować przykłady kodu źródłowego.
15. System musi generować zadania praktyczne dla użytkownika.
16. System musi umożliwiać wyświetlanie wygenerowanych materiałów edukacyjnych.
17. System musi umożliwiać ponowne wygenerowanie materiałów edukacyjnych.

Monitorowanie postępów

18. System musi umożliwiać oznaczanie ukończonych lekcji.
19. System musi prezentować postęp użytkownika w ramach ścieżki edukacyjnej.
20. System musi zapisywać historię aktywności użytkownika.

Opinie i ocena materiałów

21. System musi umożliwiać ocenę wygenerowanych materiałów edukacyjnych.
22. System musi umożliwiać dodawanie opinii do materiałów edukacyjnych.
23. System musi wykorzystywać opinie użytkownika do ulepszania kolejnych materiałów.

Integracja z modelem AI

24. System musi integrować się z wybranym modelem LLM poprzez API.
 25. System musi obsługiwać komunikację pomiędzy aplikacją a modelem AI.
 26. System musi obsługiwać błędy związane z niedostępnością modelu AI.
-

1.2. Funkcjonalności, które powinny pojawić się w aplikacji

Rozszerzona personalizacja

1. System powinien rekomendować kolejne tematy na podstawie wyników użytkownika.
2. System powinien analizować szybkość przyswajania wiedzy przez użytkownika.
3. System powinien umożliwiać wybór stylu nauki (np. teoria, praktyka, mieszany).
4. System powinien umożliwiać definiowanie celu nauki.

Rozbudowane materiały edukacyjne

5. System powinien generować quizy sprawdzające wiedzę.
6. System powinien generować podsumowania lekcji.
7. System powinien umożliwiać eksport materiałów edukacyjnych do pliku PDF.
8. System powinien umożliwiać dodawanie linków do zewnętrznych źródeł edukacyjnych.
9. System powinien umożliwiać generowanie materiałów w języku polskim i angielskim.

Interakcja użytkownika

10. System powinien umożliwiać zadawanie pytań chatbotowi AI.
11. System powinien umożliwiać reset postępu użytkownika.
12. System powinien umożliwiać zapis ulubionych materiałów edukacyjnych.
13. System powinien umożliwiać powiadomienia o zaplanowanej nauce.

Statystyki i analiza danych

14. System powinien prezentować statystyki nauki użytkownika.

15. System powinien generować raporty postępów użytkownika.
16. System powinien umożliwiać porównanie wyników pomiędzy ścieżkami edukacyjnymi.

Rozszerzenia techniczne

17. System powinien obsługiwać urządzenia mobilne.
 18. System powinien umożliwiać integrację z zewnętrznymi platformami edukacyjnymi.
 19. System powinien umożliwiać zapis danych w chmurze.
 20. System powinien wspierać wiele modeli AI.
-

2. Wymagania niefunkcjonalne (w.nf.)

Wydajność

1. System powinien generować materiały edukacyjne w czasie nie dłuższym niż 10 sekund.
2. System powinien obsługiwać jednocześnie co najmniej 50 użytkowników.
3. System powinien zapewniać płynne działanie interfejsu użytkownika.

Bezpieczeństwo

4. System musi szyfrować hasła użytkowników.
5. System musi zapewniać bezpieczną komunikację z API modelu AI.
6. System powinien chronić dane użytkowników zgodnie z zasadami RODO.
7. System powinien posiadać mechanizmy autoryzacji i uwierzytelniania użytkowników.

Użyteczność

8. System powinien posiadać intuicyjny interfejs użytkownika.
9. System powinien być responsywny i dostosowany do różnych rozdzielczości ekranów.
10. System powinien umożliwiać łatwą nawigację pomiędzy materiałami edukacyjnymi.
11. System powinien być dostępny w języku polskim.

Niezawodność

12. System powinien być dostępny przez 99% czasu działania.
13. System powinien poprawnie obsługiwać błędy połączenia z API AI.
14. System powinien wykonywać kopie zapasowe danych użytkowników.

Skalowalność i rozwój

15. System powinien umożliwiać łatwe dodawanie nowych technologii edukacyjnych.
16. System powinien umożliwiać integrację z innymi modelami AI.
17. System powinien być przygotowany do dalszej rozbudowy funkcjonalnej.

Technologiczne

18. System powinien być aplikacją internetową działającą w nowoczesnych przeglądarkach.
 19. System powinien wykorzystywać REST API lub GraphQL do komunikacji.
 20. System powinien wykorzystywać bazę danych do przechowywania danych użytkowników i ścieżek edukacyjnych.
 21. System powinien być zgodny z wybraną technologią implementacyjną pracy.
-

3. Przykładowe ścieżki edukacyjne

1. Nauka podstaw języka Java.
2. Nauka frameworka Spring Boot.
3. Nauka biblioteki React.
4. Nauka tworzenia aplikacji Full Stack.

Przygotowałem kompletną listę wymagań funkcjonalnych i нефункциональных dla projektu wraz z podziałem na:

- funkcjonalności obowiązkowe,
- funkcjonalności opcjonalne („powinny”),
- wymagania нефункциональные,
- przykładowe ścieżki edukacyjne.